

嵌入式系統

期末報告

陳信睿 *

National Taiwan University

Taipei, Taiwan

b11902047@csie.ntu.edu.tw

巫佖霆 *

National Taiwan University

Taipei, Taiwan

b11902051@csie.ntu.edu.tw

2025 年 12 月

目錄

1	簡介	2
2	專案目標、技術	2
3	資料處理	3
3.1	原始資料處理以及感測	3
3.2	校正 (Calibration)	4
4	位置預測	4
4.1	方法一：積分	4
4.2	方法二：NN-based 方法	8
5	實機測試	9
6	Ablation study	10
6.1	通訊方法	10
7	案例分析	11
8	心得	12
9	Demo 影片 (preview)	13

1 簡介

在課程中，我們認識到許多感測器：加速度計、陀螺儀以及溫度計等。我們發現 stm32l4 板子的大小很適合單手抓握，並且加速度計可以用來處理運動資料，那麼我們就思考是否可以將其製作成滑鼠裝置。

當初目標設定裝置是能在平面上移動的滑鼠裝置，不過 demo 時發現實際效果遠不如自行測試。以目前的實驗設置 (加速度計 + 不夠高的取樣率)，不適合用來處理複雜的移動軌跡，僅在單方向以及少量方向改動表現良好。

當前可行的方案有兩種，一種是基於傳統 DSP 方法的積分作法，另一種是透過 RNN 的方式學習資料和預測軌跡。前者雖然多數簡單情形可以表現良好，不過相對於後者可能處理不來複雜的情形。

老師建議我們需要將題目縮小，尋找目前進度可以解決的目標，這樣也可以讓我們避開複雜軌跡的問題。我們認為目前可以參考的運用範圍是虛擬鍵盤。虛擬鍵盤是顯示在螢幕上的鍵盤，主要用在裝置無法直接接到一般鍵盤的情況。像是電視盒子，或是在電視上看影音時，就會需要投過虛擬鍵盤輸入內容。傳統的遙控器只能透過上下左右鍵控制，非常花時間，而我們目前裝置可以處理特定方向直線移動，那麼就可以比點擊移動按鈕還快速移動到特定的鍵位上。

2 專案目標、技術

在此次專案中，我們的目標是開發 STM32 使其可以作為 Linux 桌面環境的滑鼠使用，我們將實作以下的簡化功能：

1. User Button 作為左鍵使用。
2. 以加速度計的測量結果控制鼠標位置的移動。
3. 實作相對應的 Linux 軟體作為展示用。

此專案最終需要的 STM32 開發板功能如下：

1. LSM6DSL：3D 加速度計。
2. DSP：相關訊號處理。
3. GPIO (User button)：作為滑鼠左鍵。
4. USB OTG：作為與主機進行 HID 通訊的協定。
5. Timer 及 RTOS：需要以毫秒等級的速度進行測量。
6. RNN: 手寫 inference 程式以及對應 fixed precision 版本。

3 資料處理

3.1 原始資料處理以及感測

如圖 1 在 Lsm6dsl 使用手冊中的第三十頁有提到

1. 靈敏度 (Sensitivity) 的轉換是由 `stm32l475e_iot01_accelero.c` 中處理的。我們不需要再額外套公式，透過這個介面讀到的值就可以直接對應物理意義。

2. 靜止在 $x-y$ 平面應該要測得 $(0, 0, 1g)$ ，而結果可能會有 4% 誤差。因為我們設備的移動加速度大多在 $1m/s^2$ 以下，約為 g 的 10%，這代表我們需要校正，避免誤差影響結果。

4.6 Terminology

4.6.1 Sensitivity

Linear acceleration sensitivity can be determined, for example, by applying 1 g acceleration to the device. Because the sensor can measure DC accelerations, this can be done easily by pointing the selected axis towards the ground, noting the output value, rotating the sensor 180 degrees (pointing towards the sky) and noting the output value again. By doing so, $\pm 1g$ acceleration is applied to the sensor. Subtracting the larger output value from the smaller one, and dividing the result by 2, leads to the actual sensitivity of the sensor. This value changes very little over temperature and over time. The sensitivity tolerance describes the range of sensitivities of a large number of sensors (see [Table 3](#)).

An angular rate gyroscope is a device that produces a positive-going digital output for counterclockwise rotation around the axis considered. Sensitivity describes the gain of the sensor and can be determined by applying a defined angular velocity to it. This value changes very little over temperature and time (see [Table 3](#)).

4.6.2 Zero-g and zero-rate level

Linear acceleration zero-g level offset (TyOff) describes the deviation of an actual output signal from the ideal output signal if no acceleration is present. A sensor in a steady state on a horizontal surface will measure 0 g on both the X-axis and Y-axis, whereas the Z-axis will measure 1 g. Ideally, the output is in the middle of the dynamic range of the sensor (content of OUT registers 00h, data expressed as 2's complement number). A deviation from the ideal value in this case is called zero-g offset.

Offset is to some extent a result of stress to MEMS sensor and therefore the offset can slightly change after mounting the sensor onto a printed circuit board or exposing it to extensive mechanical stress. Offset changes little over temperature, see "Linear acceleration zero-g level change vs. temperature" in [Table 3](#). The zero-g level tolerance (TyOff) describes the standard deviation of the range of zero-g levels of a group of sensors.

Zero-rate level describes the actual output signal if there is no angular rate present. The zero-rate level of precise MEMS sensors is, to some extent, a result of stress to the sensor and therefore the zero-rate level can slightly change after mounting the sensor onto a printed circuit board or after exposing it to extensive mechanical stress. This value changes very little over temperature and time (see [Table 3](#)).

圖 1: Lsm6dsl 使用手冊截圖

3.2 校正 (Calibration)

在進行校正時，我們假設以下符號：

1. 假設重力加速度大小為 g 。
2. 在靜止時，測得加速度為 $\mathbf{m} = (x, y, z)$ 。
3. g 為測量時重力加速度相對於加速度計座標的值。
4. $\mathbf{o} = (d, e, f)$ 為測量的誤差值。
5. D 為對角矩陣 $\text{Diag}(a, b, c)$ 表示各軸間的 scale。通常 D 需要考慮各軸方向不對的情況，經過靜態測試，軸方向的誤差可以省略。

理想上，測量值要有 $\mathbf{m} = g$ 。但由於誤差以及各軸間不同的 scale，實際關係如下：

$$\mathbf{m} = D\mathbf{g} + \mathbf{o}。$$

校正的目標則是找到 D 以及 $\mathbf{o}$ 。我們有

$$\begin{aligned} |\mathbf{g}|^2 &= |D^{-1}(\mathbf{m} - \mathbf{o})|^2 \\ &= \left(\frac{x-d}{a}\right)^2 + \left(\frac{y-e}{b}\right)^2 + \left(\frac{z-f}{c}\right)^2 \end{aligned}$$

因為 $|\mathbf{g}|^2$ 為常數，我們只要找到六組靜止實測得的 $\mathbf{m} = (x, y, z)$ 即可。這個想法其實就是 Dt0059 [Vit] 這個設計中使用的作法，只是在數學上的表達稍微不一樣。

現在只要測量多組 (x, y, z) ，然後就可以透過解線性方程組來得到 D 以及 $\mathbf{o}$ 。此外如果採樣更多資料，則可以透過線性回歸來得到 D 以及 $\mathbf{o}$ 。接下來，我們用 $\mathbf{a}$ 來表示校正過後的加速度資料： $D^{-1}(\mathbf{m} - \mathbf{o})$ ，並且僅保留 x, y 軸資料。

4 位置預測

4.1 方法一：積分

模型. 如果我們直接使用二次積分去計算板子實際移動距離，會遇到以下情況：

1. 若校正後還是有一定的誤差，將導致靜止時速度不斷上升，積分結果不會收斂。
2. 移動週期不是固定的，需要想辦法減少誤差累積造的速度。
3. 因為要求為即時低延遲，資料不能先收集後靜態處理，且資料為非線性的，濾波器的選擇至關重要。
4. 判斷停下、轉換方向、繞圈等簡單的行動模式。
5. 往一個方向移動時，因為晃動造成垂直方向也跟著移動。

考慮以下描述形式：

$$\begin{bmatrix} p_x^{t+1} \\ p_y^{t+1} \\ v_x^{t+1} \\ v_y^{t+1} \\ a_x^{t+1} \\ a_y^{t+1} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & \Delta t & 0 & \frac{1}{2}\Delta t^2 & 0 \\ 0 & 1 & 0 & \Delta t & 0 & \frac{1}{2}\Delta t^2 \\ 0 & 0 & 1 & 0 & \Delta t & 0 \\ 0 & 0 & 0 & 1 & 0 & \Delta t \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}}_{\text{Transition Matrix}} \begin{bmatrix} p_x^t \\ p_y^t \\ v_x^t \\ v_y^t \\ a_x^t \\ a_y^t \end{bmatrix} + \underbrace{\begin{bmatrix} \frac{1}{2}\Delta t^2 & 0 \\ 0 & \frac{1}{2}\Delta t^2 \\ \Delta t & 0 \\ 0 & \Delta t \\ 1 & 0 \\ 0 & 1 \end{bmatrix}}_{\text{Noise Gain Matrix}} \begin{bmatrix} o_x \\ o_y \end{bmatrix}$$

通常會直接定義 $\Delta t = \omega^{-1} = (2\pi f)^{-1}$ ，其中 o_x, o_y 為雜訊，一般來說會使用 Gaussian Distribution 去估計，不過在我們的觀察下，可能會是不小的靜止誤差再加上 Gaussian Distribution。

如果排除靜止誤差的部分，若 o_x, o_y 足夠小，那麼誤差其實不會影響很大。但加上靜止誤差後，每經過一秒，誤差會顯示出來。假設取樣離散化數據後積分上沒有過多誤差值，那麼最重要的部分就是偵測停止狀態。這邊我們引入一個偵測模組負責處理停止狀態偵測：

ZUPT. ZUPT (Zero velocity update)，用在慣性感測器 (IMU) 上來檢測物體是否停止，並在停下時做校正。通常用在計步器或是組合導航 (INS, GPS)。

這些技術的共同特徵是會透過各種 IMUs (加速度計、陀螺儀、磁力計) 來做位置、速度、角度的估計與校正。其中 ZUPT 的定位就是用來校正 sensor 使用過程中資料的累積誤差 (drift)。我們參考了 [Sko+10]，這篇文章說明 ZUPT 如何應用在計步器上，想用類似方法應用在加速度計上。

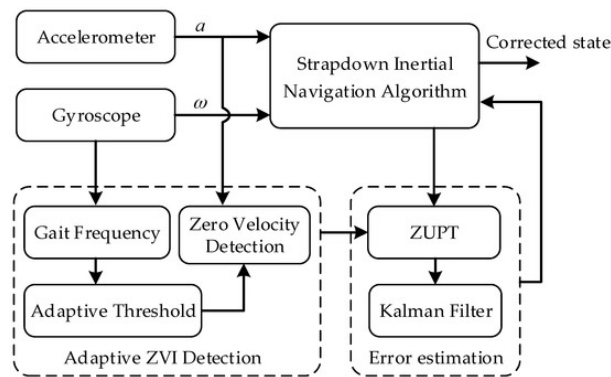


圖 2: Example Block Diagram used for pedestrian nagivation

在使用 ZUPT 做即時校正前必須要能夠偵測 IMU 靜止，那麼有兩種方法可以切入：

1. Variance: 當 IMU 靜止時，其訊號應該要相當穩定 (只有雜訊波動)，

$$\frac{1}{N} \sum_{k=1}^N \|a_k - \bar{a}_k\|^2 < \gamma$$

2. Magnitude: 當 IMU 靜止時，其總加速度大小應該要等同 g ，

$$\frac{1}{\sigma_a^2 N} \sum_{k=1}^N (\|a_k\| - g)^2 < \gamma$$

如過要同時考慮，一般工程思維就是直接做 “and expression”，而以統計為出發點，

$$\frac{1}{N} \sum_{k=1}^N \frac{1}{\sigma_a^2} \left\| a_k - g \frac{\bar{a}_k}{\|\bar{a}_k\|} \right\|^2$$

不過我們為了減小資料處理複雜度，已經先把 z-axis 的資料丟掉了，所以合適的方法是使用 “and” 再去調整閾值 (threshold)。

Filter. 通常 ZUPT 方法會搭配 Kalman filter 使用。不過 Kalman filter 比較適合用在靜態且線性的資料，在像是滑鼠的裝置上效果不會很好 (會一直預測失敗大幅修正 covariance)。

針對非線性的資料，比較適合透過像是 1€ filter 中的 Exponential smoothing 處理。最簡單、低成本的方法就是使用 sliding window 加上一些 heuristic 的規則處理。缺點就是有不少超參數要調整，需要花很多時間做 grid search。

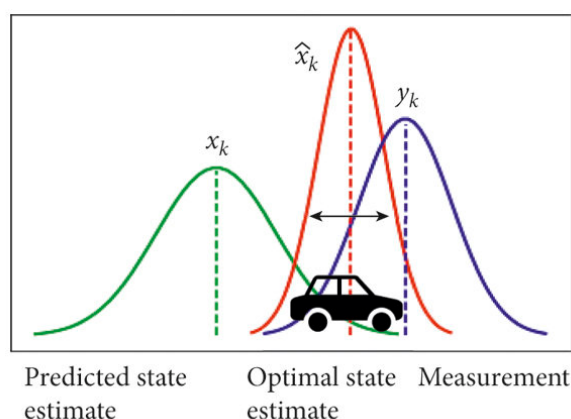


圖 3: kalman filter

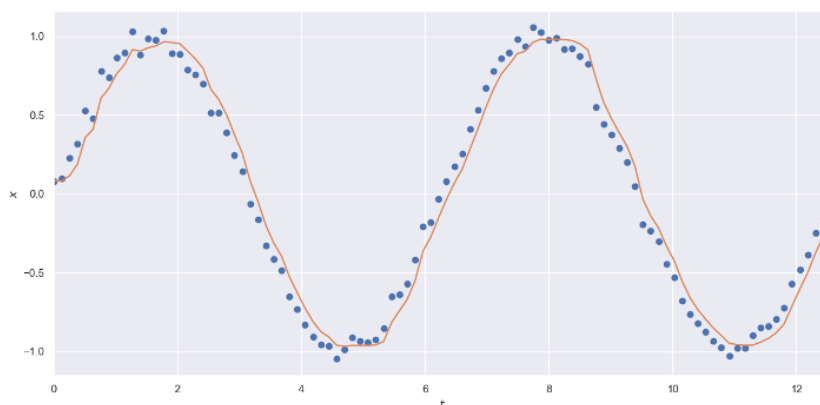


圖 4: 1€ filter

最終設計. 爲了在短時間內先有一個能用的方案，最簡單的方案是使用 ZUPT 加上 sliding window 的實作方法。新增 frame count，記錄符合停止條件的連續 frame 數量，超過閾值才會將速度歸零。alpha 使用來更新 bias (draft) 的權重:

$$bias_{update} = (1 - \alpha) \cdot bias_{original} + \alpha \cdot a$$

實驗結果以下參數表現不錯 (取樣率 100 Hz):

1. window size: 8.
2. variance threshold: 0.16.
3. magnitude threshold: 1.6.
4. frame count: 3.
5. alpha: 0.15.

這樣的設置可以在 IMU 停下來 0.1 秒時偵測出來，並且 frame count 設定爲 3，可以減少雜訊擾動導致誤判邊界的機會。而引入 Kalman filter 和 1€ filter 嘗試改進二次積分算法後，某些直線或是曲線情況得以改進，不過複雜一些的情況表現依舊沒有改善。下面列出初步實驗所使用的參數。

Kalman filter 所使用的參數：

1. R_Base: 0.06.
2. Q: 0.002.

而以下爲 1€ filter 使用的參數：

1. min cutoff frequency: 1.2.
2. beta: 0.04.

此外共同的參數，用在模擬物體移動使用的衰減參數，也就是摩擦力爲 0.998。

加上 filter 設計的困難點在於，不好去控制 cut-off frequency，因爲裝置在移動的晃動、位移大小都會影響到最適合的 cut-off frequency，一旦這個 threshold 沒設好，相關的 ZUPT 設計也不太好處理。我們認爲這部分只能靠更細緻的訊號分析與處理技術才能有效推得最佳參數，目前我們知識儲備有限，不太會設計好的 filter。

事實上，多數 ZUPT 搭配 filter 的案例都是放在人體上的監測裝置，如計步器等。這些裝置的共同點是訊號的週期會拉很長，誤差容忍相對來說也比較大，且人體移動訊號很容易分析出高頻與低頻區域，那麼 DSP 的處理也比較能夠有效地將雜訊去除。而在如滑鼠的裝置上，目前我們沒有找到相關的工作也用這個方法，因爲基本上 optical sensor 配上高取樣率效果已經遠比加速度計爲主的裝置好很多。

4.2 方法二：NN-based 方法

方法與損失函數設計. 我們使用 RNN 以及其變種 (GRU, LSTM) 等等來預測現在 STM 32 開發板的移動速度：

$$\mathbf{v}_t = \alpha f_{\theta}(\mathbf{a}_t)$$

接下來就可以透過 $\mathbf{v}_t$ ，來決定輸出的 HID Report dx , dy 。我們目前訓練模型輸出的 $\mathbf{v}_t$ 表示當前的速度，而 α 則是板子加速度計的單位以及公分間的轉換。

我們最初設計的損失函數如下：

$$\mathcal{L}_p := \left\| \sum_t \alpha \mathbf{v}_t \Delta T - \mathbf{p} \right\|^2$$

這個設計的目的，是希望模型預測出的速度序列 $\mathbf{v}_t$ ，在經過積分後，能夠重建板子實際的位移 $\mathbf{p}$ 。然而，用這個損失函數單獨訓練出來的模型，其生成的路徑往往和真實的加速度 $\mathbf{a}_t$ 不一致，結果看起來像是一條隨機的軌跡。

在觀察資料後我們發現，對於某些動作較簡單的樣本，直接對加速度做兩次積分所得到的位移其實表現不錯。基於這點，我們對其中一部分的資料進行一次積分，得到每個時刻較可靠的速度估計 $\tilde{\mathbf{v}}_t$ 。接著，我們在這些資料上加入額外的限制，使模型的預測速度與這些估計更接近：

$$\mathcal{L}_v = \sum_t |\tilde{\mathbf{v}}_t - \alpha \mathbf{v}_t|^2。$$

在原本的 $\mathcal{L}_p$ 上加入 $\mathcal{L}_v$ 之後，模型的表現雖然有所改善，但仍然無法學到一般性的預測能力。我們發現，模型產生的路徑依然沒有符合由加速度 $\mathbf{a}$ 所推導出的物理關係

為了強化模型對加速度約束的遵從，我們引入了以下的加速度損失：

$$\mathcal{L}_a = \sum_t \sigma \left(\left| \mathbf{v}_t - \mathbf{v}_{t-1} - \left(\frac{\mathbf{a}_t + \mathbf{a}_{t-1}}{2} \right) \Delta T \right|^2 - \tau \right)，$$

其中 τ 為一個設定好的閾值， σ 則是 GeLU 或是 ReLU 用以抑制小於閾值的誤差、只強調較大的物理偏差。最終我們訓練的模型以

$$\mathcal{L} = w_p \mathcal{L}_p + w_v \mathcal{L}_v + w_a \mathcal{L}_a$$

作為損失函數。

資料增強. 我們最終蒐集 128 筆的資料，因此對於訓練一個複雜的 RNN 容易導致 overfit，我們決定使用以下資料增強的方法：

1. 將加速度的序列，隨機在序列中加上一些小的 Gaussian noise。
2. 將加速度的序列的 x 軸與 y 軸進行翻轉，這樣可以得到四倍的資料。

在加上資料增強後，我們發現原本簡單的 RNN 無法將 loss 降低，因此在這之後我們採用更複雜的 GRU。雖然最終的 checkpoint 沒有使用這個技巧，但是這也是我們曾經採用過的方法。

RNN 變體以及 Quantization. 我們目前嘗試了兩種 RNN 變體，以下為一般的 RNN：

$$\begin{cases} \mathbf{h}_{\ell,t} = \tanh(U_{\ell}\mathbf{h}_{\ell-1,t} + V_{\ell}\mathbf{h}_{\ell,t-1} + b_{\ell}) \\ \mathbf{v}_t = W\mathbf{h}_{L,t} + b_{\text{out}} \end{cases}$$

其中 $\mathbf{h}_{0,t} = \mathbf{a}_t$ 為輸入，而 $\tanh$ 也可以選擇其他 Non-linearity。

前面的模型似乎無法有效處理我們資料的複雜度，因此我們接下來嘗試了進一步的 GRU。

$$\begin{cases} \mathbf{z}_t = \sigma(W_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + b_z) \\ \mathbf{r}_t = \sigma(W_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + b_r) \\ \tilde{\mathbf{h}}_t = \tanh(W_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + b_h) \\ \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \end{cases}$$

在 GRU 中引入了 Update gate 以及 reset gate 的概念，這可以幫助機器學習到什麼時候需要把速度 reset 成 0。

在課堂中，我們學到 CMSIS 的 DSP 函式庫，包含一些矩陣的操作，或許可以透過這個函式庫進行推論。不過 ARM Cortex-M4 架構支援得指令集不包含 NEON 的向量運算，實作上會 fallback 回原始算法。不過 DSP 有使用 q15 加速的實作，使用上需要先將模型轉換成 q15 fixed point。

在引入 q15 fixed point 前，我們得到的模型會需要進行 quantization。想法大致上是將 NN 中的每個矩陣 V ，找到一個倍數 s ，使得 sV 中每個係數介在 -1 到 1 之間，然後就可以把每個數對應到一個最接近的 q15 有理數。這樣就可以用新的 q15 矩陣以及倍率 s 來進行計算。

5 實機測試

實作. RTOS 的部分使用三個 threads、一個 timer、多個 lock objects 控制順序：

- Thread1: 從加速度計讀取資料，由 Timer 觸發
- Thread2: 處理加速度計資料並且預測位移。
- Thread3: 傳送 HID report 到 host 端。
- Interrupt handler: 處理按鈕相關 callback。
- Timer: 目前設定取樣率為 100 Hz。每隔 0.01s 叫 thread1 讀資料。

在實作上，我們注意到：

1. USBD_HID 相關函式庫已經有覆寫了 HAL_GPIO_EXTI_Callback，我們需要找到這個函式位置並且實作按鈕相關函式。

2. 多個 threads 間有順序關係，且無 overlap 工作的需求。效能上將任務集中在一個 thread 完成為最佳。

因此我們改良成以下作法：使用一個 thread，依序進行取得加速度計資料 → 進行資料處理 → 最後則送到 USB port 上。Interrupt handler 則會處理按鈕的邏輯。至於 timer，只要確保 thread 在 0.01s 內可以完成工作並使用 osDelayUntil 即可。

RNN 實作。 因為 CMSIS library 只有提供 CNN 相關的 library，所以除了共同有的 matrix 操作以及四則運算以外，我們都要手動去實作。目前實作了三個方案：

1. Naive: 用基本語法直觀實作。

2. Matrix: 引入 library 中的 matrix 函數計算。

3. q15: 使用 q15 型態計算，雖然有 q15 + matrix 的函式，不過並沒有針對 overflow 的情形作處理，所以只能逐行做 inner product 成 q63 再轉回 float 計算每一項。

因為 RNN 的架構與 GRU 的架構有些許不同，所以我們在這邊各實作一份，所以總計有六種方案被我們實作出來。

注意到 GRU 在計算過程中會需要使用 sigmoid，而目前的計算方法沒有任何優化，所以必定會使用到浮點數的 exponential 計算。當使用的參數 layer = 3 與 hidden size ≥ 20 時，原始只用浮點數計算的方法無法在 1ms 內計算完，不過改使用 q15 後情況有大幅改善。風險就是輸出結果的正確性有待 profiling，進一步的分析 activation。

6 Ablation study

6.1 通訊方法

在目前裝置上可用的通訊方法有很多：USB, BLE, Wifi, UART。以 HID 來說主要會考慮 USB 和 BLE 的方案，而少數情況會透過 wifi (Remote Desktop Protocol)，極少數嵌入式裝置會用 UART (baud rate 等可能會有對齊上的問題)。

另外在 HW7 中，因為實驗要求我們定時一次發送大約 512 bytes 的資料，而我們使用 BLE 傳送時，遇到 BLE 在連續大量傳送封包時會卡 buffer 的問題。並且藍芽裝置睡著的情況在 PC 界是個很常見且難解的問題。儘管課堂上我們花很多時間在學習、實作 BLE 的內容，但經過這些考量，以合理性來說，BLE 會帶來更多潛在的問題使得裝置穩定性變差。

而 Wifi 的部分延遲不低且 host 會需要額外模組或是通訊協議設定才能使用。對於我們的實驗目標，會希望裝置的移動可以即時的被處理顯示出來。

綜上，使用 USB HID 才是最有效率的方法，唯一的缺點就是需要額外接一條線使用，而當 host 端給的電壓不足，還需要另外一條線供電。

7 案例分析

我們實驗過不同大小的模型，hidden size 為 10, 12, 20，我們發現在 $\text{hidden size} \leq 12$ 時不太能練不起來，而選擇 20 會遇到 overfit 的問題。

分析. 圖 5 為我們 RNN 預測以及使用 ZUPT（未使用 filter）的軌跡，可以看到在直線上都有不錯的表現。

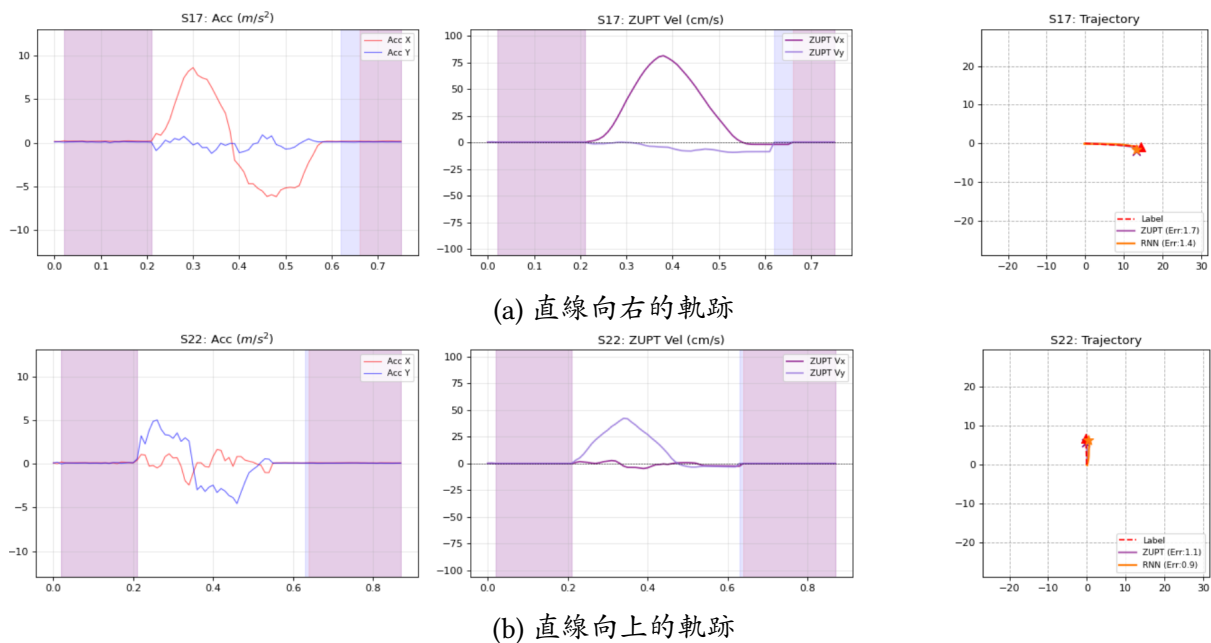


圖 5: 直線軌跡

圖 6 為簡單繞圈的範例，此時 RNN 的表現已經不錯，而 ZUPT 對於這種長度的 sequence 會出現漂移的現象。然而 RNN 在實機測試時存在更嚴重的 overfitting 現象。

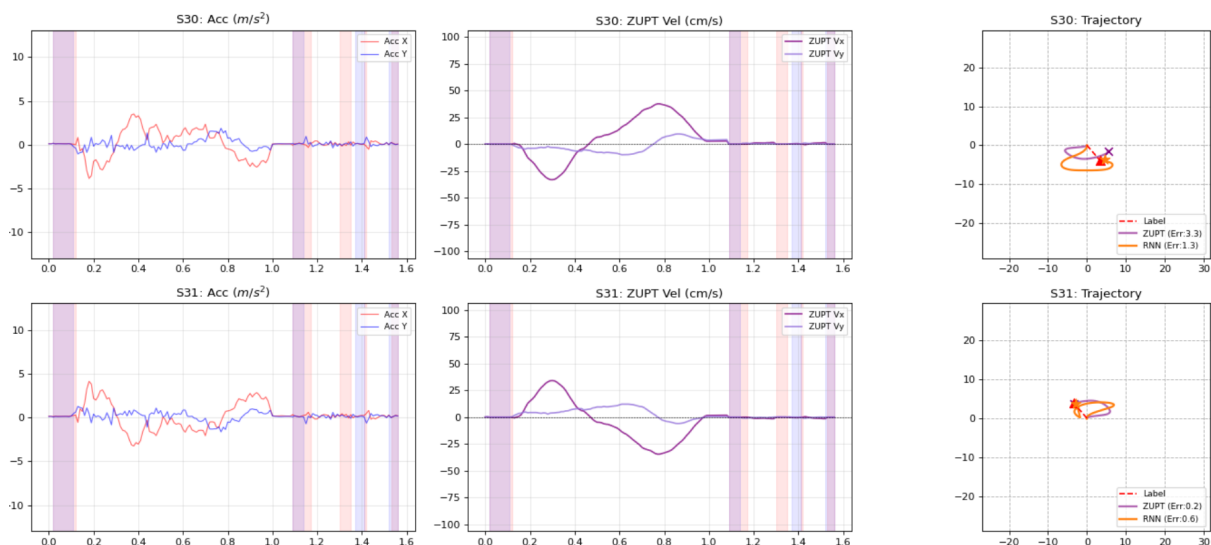


圖 6: 簡單繞圈

最後圖 7 則是更複雜繞圈的狀況，此時 RNN 也可將誤差校正回來，然而 ZUPT 則會有更嚴重的誤差累積。儘管如此，RNN 的作法雖看似非常有力，但仍被嚴重的 overfitting 讓表現在實際操作便得不可用，反而是 ZUPT 對於一般短暫的手勢操作等，可以有更好的表現。

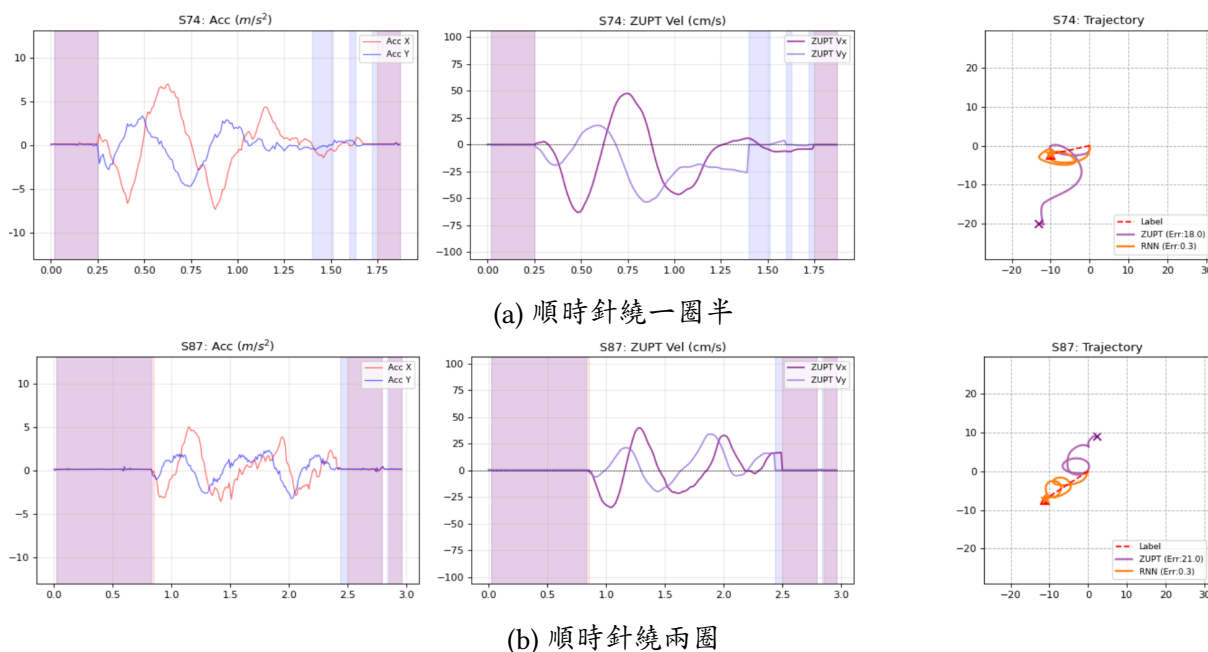


圖 7: 複雜繞圈

8 心得

在一開始的設計中，我們有考慮過使用多一些 RTOS 以及 BLE 的功能，因為這些都是上課提過的内容。不過在經過種種改良下，我們發現如果要讓這個主題執行的穩定且高效，有些功能實屬多餘。

最終使用的技術大約只有加速度計、DSP 與 fixed precision 加速技巧跟課堂上有關。取代 BLE 的部分是引入板子上另有的 USB OTG 功能。此外 RTOS 本質上適合同時處理多種任務，但我們發現 USB HID 協議的設計上我們可以將多種任務和唯一，可只使用單個執行緒就完成所有事情，這在排程以及效能上都比強硬應用 RTOS 功能還好很多。

目前我們在專案上的重心比較偏向數值處理以及分析，比較少比重在應用或是解決實際問題上。也許最後效果沒有很好，但可能算是探索了一大部分的内容。

如果以後我們或是有人會需要再做類似的題目，我們會建議不要把實驗的環境限制在平面上，相對的加速度計應該比較適合用在三維空間上，搭配陀螺儀進行裝置旋轉角度或是角速度的計算。

這麼想的原因是，我們花很多時間在處理加速度以及移動速度上的問題，而且到後面透過 filter 也沒有顯著改善，除了在大幅度移動的狀況，其餘小範圍表現在加上

filter 後可能還會比原本更差。然而因為適合移動的範圍大，這也讓我們更容易想拿起裝置回到原點再進行下一次移動，在拿起移動過程又會因此多移動，或是往更差的方向跑，這樣根本就是本末倒置。

但換個念頭如果是用三維的選轉角度看待，那麼就可以以恆定速度去處理 (譬如旋轉超過一定角度給予對應的恆定速度)，從而避開二次積分和誤差累積的問題，並且也不太會有移動到原點的額外需求。

9 Demo 影片 (preview)

1. 影片連結一。
2. 影片連結二。

參考資料

- [Sko+10] Isaac Skog et al. “Zero-Velocity Detection—An Algorithm Evaluation”. In: *IEEE Transactions on Biomedical Engineering* 57.11 (2010), pp. 2657–2666. DOI: 10.1109/TBME.2010.2060723.
- [Vit] Andrea Vitali. *DT0059 Design tip. Ellipsoid or sphere fitting for sensor calibration*. https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://www.st.com/resource/en/design_tip/dt0059-ellipsoid-or-sphere-fitting-for-sensor-calibration-stmicroelectronics.pdf&ved=2ahUKEwjqzdbT4LCRAxVu2TQHHTVREqYQFnoECAwQAQ&usg=AOvVaw3vx D1m08W9ydBmEgkXz5MD.